HCP_VPCG Plugin User Guide

Introduction

The HCP_VPCG plugin adds four additional nodes and c++ code to the unreal engine PCG plugin suite of nodes that has the ability to generate a hexagonal close packing (HCP) 3D matrix grid, and 2D hexagonal surface grid of points. The V in HCP_VPCG stands for voxel, as the initial intention of this plugin project was to create a plugin to perform a hcp voxel generation and capable processing plugin to use within Unreal Engine 5.7.

In 3D space utilising the Cartesian coordinate system, a 3D matrix grid is of a cubic form where each cube can be considered as a voxel of cubic shape. For any one of these cubic voxels, the distance between its centre and that of every neighbour around it is not the same. However, a 3D space that does have a structure where the centre of any location in that space is of equal distance to all its neighbours is that of a hexagonal close packed spheres of equal radius, where the geometric shape of a Trapezo-rhombic dodecahedral can be arranged in such a manner that it forms a space filling geometry that can be utilised as a voxel.

It is of the view that a HCP sphere structure is superior than a cubic structure for use in procedural generation of geometric shapes, to simulate 3D environments and for cellular automata.

Unfortunately Unreal Engine proved to be limited in terms of its management and allocation of computer memory resources to be able to implement of what can be achieved in such an implementation of this plugin to its full potential. In particular the ability to use HCP for automata and simulation of large datasets. However, Unreal Engine is a game engine and is not designed or implemented for such applications, but in respect to its purpose and design, this HCP_VPCG plugin can be useful and is thus created and implemented to be in addition to the existing Unreal Engine PCG plugin system.

Further information of hexagonal close packing

Information on what hexagonal close packing structure is can readily be found on searching the web.

For the purposes of how hexagonal close packing is defined and giving greater insight on the code for this plugin is given within the pdf document file vw_HCP_voxel_draft01.pdf included in the directory of this document. The vw_HCP_voxel_draft01.pdf document has more detail and information than required as it is the documentation of the github hexagonal close packing C++ project from which the code of the HCP_VPCG plugin originated from, and was adapted to be able to be used in the unreal engine PCG graph editor.

Installation

Installation of the HCP_VPCG unreal engine plugin is similar to any unreal engine plugin. The plugin source and other files of the plugin contained within the HCP_VPCG Plugins directory are copied to the unreal engine project Plugins directory that one wishes to use the plugin.

Upon starting the unreal engine project for the first time that the plugin is to be used, unreal engine may ask to build the HCP_VPCG plugin. Click on the yes button to build the plugin. Then once the unreal engine project is up and running, go to the edit menu within the unreal engine editor interface and select plugins. Once the plugins selection list appears, type PCG in the search bar. Then select both the PCG Content FrameWork and HCP_VPCG plugins, and restart the unreal engine project.

It may also be possible that instead the building of the HCP_VPCG plugin is performed on restarting the unreal engine and not the initial starting process outlined above. In either case, once the unreal engine project is successfully restarted, the plugin can be used.

Access Within Unreal Engine 5.7

The HCP_VPCG plugin nodes are accessed in the same manner as the default unreal engine PCG graph nodes within the PCG graph editor. Open up a new or existing PCG graph editor and within the graph editor press the right mouse button to display the node selection list widget. In the search bar type hcp, and all the four available HCP_VPCG nodes will be displayed.

01 The HCP_VPCG Nodes

Create HCP voxel points grid

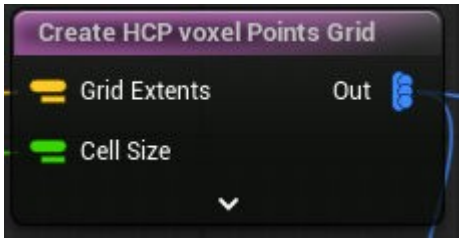

Fig 01.0.1

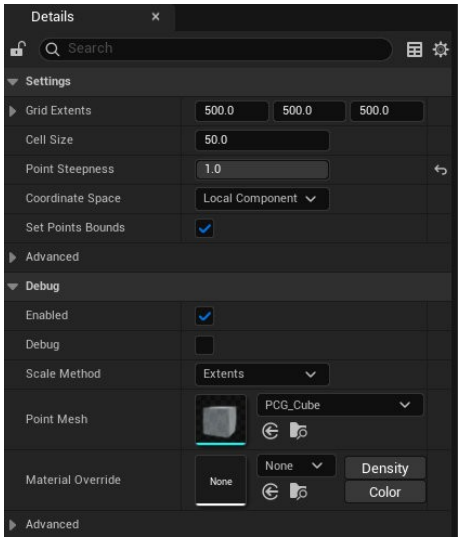

Fig 01.0.2

The Create HCP voxel Points Grid node is similar to the default PCG “Create Points Grid“ node, but if one looks at the HCP voxel Points Grid node details panel (**Fig 01.0.2**), one will see that there is only one grid cell size and other panel entry differences. This is because HCP voxel matrix grids must have a uniform grid cell size in all directions as they represent the 3D space of a sphere that can fit into such a voxel. Another difference is that no culling of the HCP_VPCG is permitted.

Thus upon selecting the HCP voxel Points Grid node, the user need to concern themself with defining the grid extents in each of the xyz Cartesian axis, and the grid cell size. These inputs can be defined as the node inputs from outside the node details panel by expanding the node input parameters and linking a float3 vector variable and float variable to the grid extents and cell size default node pin inputs that the unreal engine has generated in the PCG editor. (**Fig 01.0.1**).

Upon creation and defining the grid parameters for this node, an appropriate HCP grid of point data representing the central points of the HCP voxel matrix grid will be created.

HCP Filter Odd Even Levels

Fig 01.0.3

Upon creation of a HCP Voxel points grid, each grid point displayed as a voxel is of the geometric shape of a Trapezo-rhombic dodecahedral. As described in the document vw_HCP_voxel_draft01.pdf, the hcp voxel grid is defined as a grid of layers of hexagonal grids in the direction of the z axis displaced in a consistent manner, and the geometry of a Trapezo-rhombic dodecahedral at each grid point forms a solid shape with no gaps of this grid if each odd layer is rotated 60 degrees on the x axis in respect to that of the even layer.

This is what this node does. A HCP grid generated by the **Create HCP voxel points grid** node or a derivative of is the input into this node. The **HCP Filter Odd Even Levels** then filters or separates this grid node data into two output streams of grid point data that are the even and odd levels of this input hcp grid.

The output even level can then be used to display a static mesh of the Trapezo-rhombic dodecahedral geometric shape for the even levels, and the output odd level can then be used to display a static mesh of the Trapezo-rhombic dodecahedral geometric shape rotated 60 degrees for the odd levels of the hcp points grid.

Note: For this filter to operate correctly, it must have its grid extent and grid cell size input values to be the same as that of the input HCP Grid.

Note: To display the Trapezo-rhombic dodecahedral geometric shape correctly, two static meshes called hcp_even and hcp_odd are supplied in the Content/Meshes directory that can be copied/loaded into the unreal engine projects Content directory.

HCP Grid Surface Filter

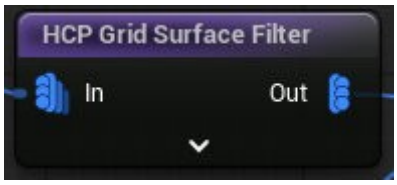

Fig 01.0.4

The Grid Surface Filter node has the purpose to filter and pass out the hcp point data that defines a 3D surface that a hcp voxel grid represents.

This is done by the value of its density attribute being greater than zero representing a hcp voxel cell being part of a solid structure, and a value of zero representing a hcp voxel cell not being part of a solid structure. This filter then performs a procedure to examine all of the neighbours of a particular hcp voxel cell (ie hcp point data) and if that voxel cell is itself has a density value greater than zero, and has any neighbour that is of a value of zero, then that hcp grid voxel point data is passed through this filter.

If the voxel being examined has a density value of zero, it is not considered as a surface voxel and is not passed on and out.

If the voxel being examined has a density value greater than zero, and all of its neighbour voxel cells also have a density value greater than zero, then it is an interior solid voxel cell and is not passed on and out.

Note: For this filter to operate correctly, it must have its grid extent and grid cell size input values to be the same as that of the input HCP Grid.

Note: The HCP even and odd level data is lost and thus the output data cannot be displayed as Trapezo-rhombic dodecahedral geometric shapes.

Create HCP Surface Points grid

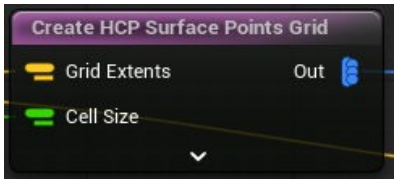

Fig 01.0.5

The Create HCP Surface Points Grid node performs the same function as the Create HCP voxel Points Grid node, but creates only the first even layer of a HCP voxel Points Grid. Thus a 2D hexagonal grid is created with the Grid Extents being of a float2 type vector defining the x and y grid extents. As with the HCP voxel Points Grid, the grid cell sizes must be of one single value.

02 Examples of usage

Copy Example files

In the HCP_VPCG directory is a directory with the Label Content.

Once it is established that the HCP_VPCG plugin has been successfully implemented in the unreal engine project that it is to be used in, be sure to close the unreal engine project and copy the sub-directories Meshes and PCG_Graphs into the unreal engine projects Content directory. The files copied are in the Meshes Directory the static maeshes hcp_odd and hcp_even of the rapezo-rhombic dodecahedral geometric shapes for odd and even hcp 3D grid levels, and PCG graphs to demonstrate the use of the HCP_VPCG plugin.

Then restart the unreal engine project to gain access to this copied content.

Create a HCP point grid

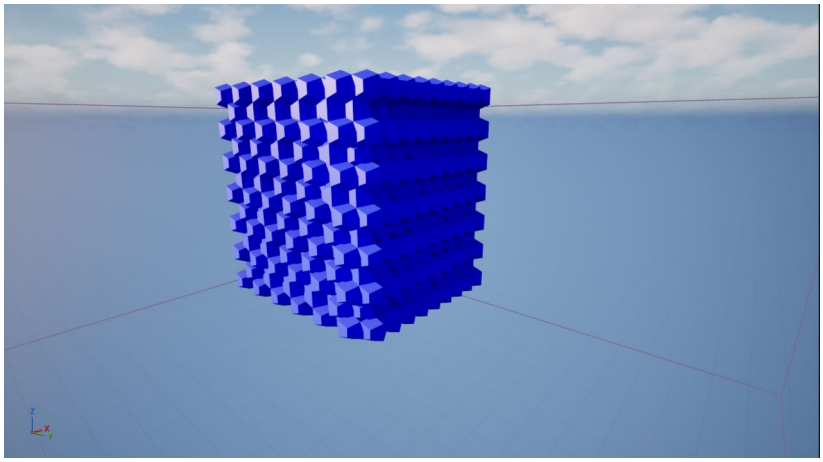

Fig 02.0.1

The most basic first implementation of the HCP_VPCG plugin is to create and display a hcp grid of points as in **Fig 02.0.1**. If one had copied the files as given in **Copy Example files** above, then the user upon opening their unreal engine project need only to use an existing level or create and define a new one with the desired lighting to add the copied PCG graphs into.

To create a display of a hcp voxel grid similar to **Fig 02.0.1**. a PCG graph file called HCP_Volume should exist within the PCG graphs directory of the unreal engine project Contents directory. Examine this PCG graph in the unreal engine PCG graph editor to make sure it has the same appearance and node links as that given in **Fig 02.0.2**.

It may be that the Create HCP voxel points grid node is missing. If this is the case, then press the right button of the mouse and from the node selection widget, search for hcp voxel points grid and select it to add the node to the PCG graph editor. Connect all of the node links as given in **Fig 02.0.2**.

Next double click on the HCP voxel Volume Display SG sub graph, and a new PCG graph similar to **Fig 02.0.3** should appear. Again, if the HCP Filter Odd Even Levels node is not displayed, perform the same task as with the Create HCP voxel Points Grid to create the same PCG sub graph as in **Fig 02.0.3**.

Once it is confirmed that the PCG graph HCP_Volume is the same as given in **Fig 02.0.2**. and **Fig 02.0.3** drag and drop the PCG graph HCP_Volume into the level, and once done so, a display similar to **Fig 02.0.1** should be present.

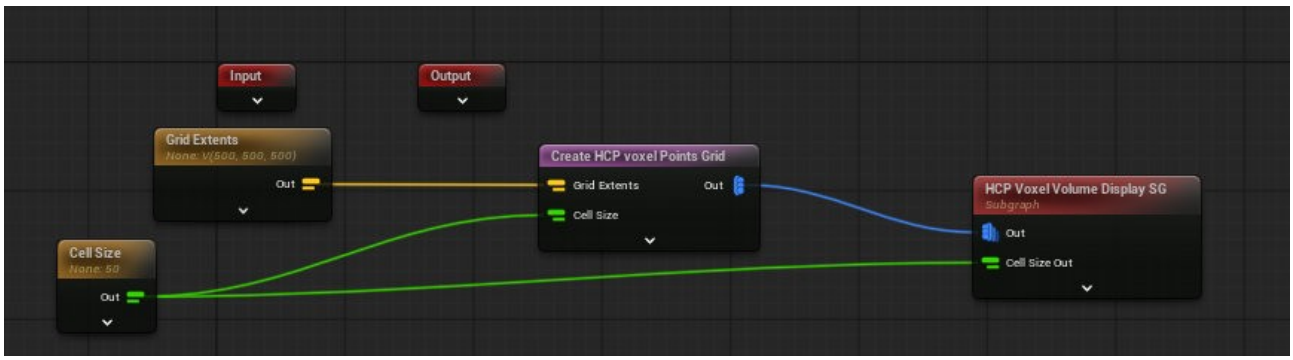

Fig 02.0.2

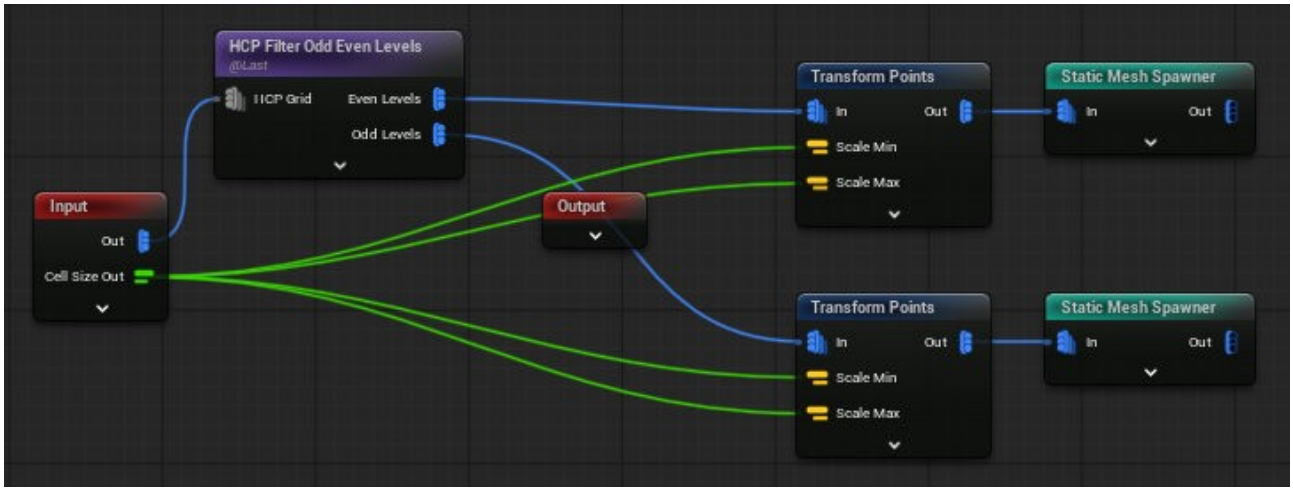

Fig 02.0.3

If starting from scratch, then the PCG sub graph of **Fig 02.0.3** and the PCG graph of **Fig 02.0.2** need to be created. The Grid extents node is a float 3 vector node and the CellSize node is a float node. The given grid extents used are 500,500,500, and cell size 50. By changing these values, the user can see how this generated hcp voxel grid changes and what limits the unreal engine has on generating such a point grid.

Create a HCP point grid interaction operation

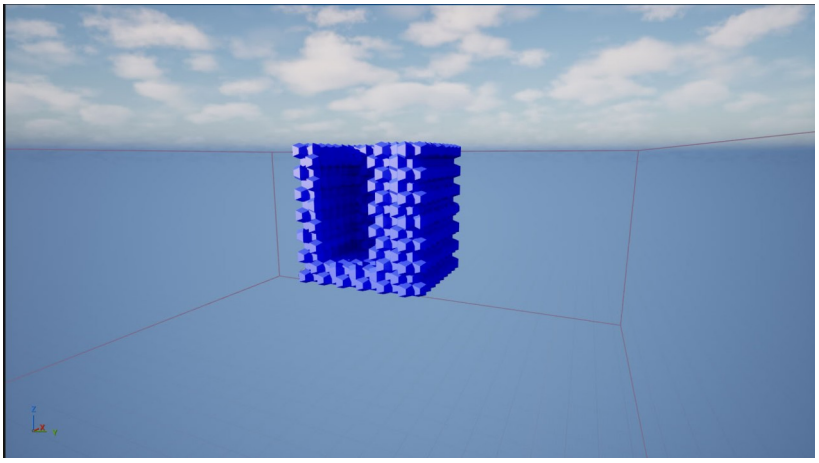

Fig 02.1.1

One of the goals of creating and implementing the hcp voxel grid plugin for the unreal engine was to be able to overlap two hcp voxel grid and perform boolean and other interacting operations upon them to produce a third resultant hcp voxel grid. Such an interaction operation is illustrated in **Fig 02.1.1** where a boolean difference function is performed on two hcp voxel point grids by creating an unreal engine PCG graph illustrated in **Fig 02.1.2**.

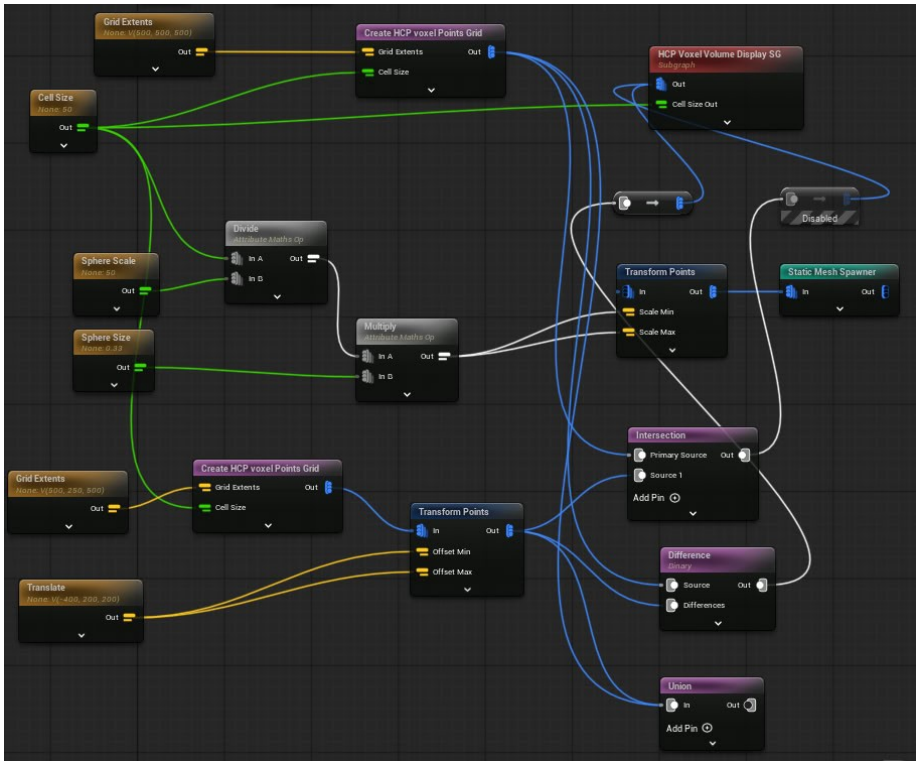

Fig 02.1.2

A PCG graph file called HCP_Volume_test should exist within the PCG graphs directory of the unreal engine project Contents directory. Examine this PCG graph in the unreal engine PCG graph editor to make sure it has the same appearance and node links as that given in **Fig 02.1.2**.

It may be that the Create HCP voxel points grid node is missing. If this is the case, then press the right button of the mouse and from the node selection widget, search for hcp voxel points grid and select it to add the node to the PCG graph editor. Connect all of the node links as given in **Fig 02.1.2**.

If the HCP voxel Volume Display SG sub graph node is not present, perform the same task as with the HCP voxel points grid node to have this node defined. Make sure the HCP voxel Volume Display SG sub graph is the same as in **Fig 02.0.3.** If it is not, then perform the task as outlined for creating a voxel point grid above in the section **Create a HCP point grid**.

Once the user has a PCG graph similar to **Fig 02.1.2**, drag and drop this PCG graph into the unreal engine level displayed in the editor.

If the hcp grids do not overlap, the display will not resemble that in **Fig 02.1.1**. To make sure the hcp grids overlap, select the HCP_Volume_test PCG graph in the outliner of the unreal engine editor, and make sure that within the details panel of the unreal engine editor that the location values within the transform drop down have x,y,z all set to a value of zero.

The complexity of **Fig 02.1.2** can be reduced by removing the central nodes that are present to display the point grid interaction as spheres rather than as a Trapezo-rhombic dodecahedral geometric shape.

It is critical that the two hcp grids be of the same cell size for this interaction operation to be a success as well as the final display grid. Thus a single Cell Size float variable is defined and linked to the two create HCP voxel point grid nodes and HCP voxel Volume Display SG sub graph.

To offset the second hcp voxel grid from the first, a PCG transform points node is used. The two hcp voxel grids are then used as inputs to the PCG Difference node which is then the input grid data to be used by the HCP voxel Volume Display SG sub graph.

Another boolean interaction or operation of union of the two hcp voxel grids can be illustrated by instead of having the Difference node linked to the HCP voxel Volume Display SG sub graph, to have the union node linked instead.

HCP Grid Surface Filter

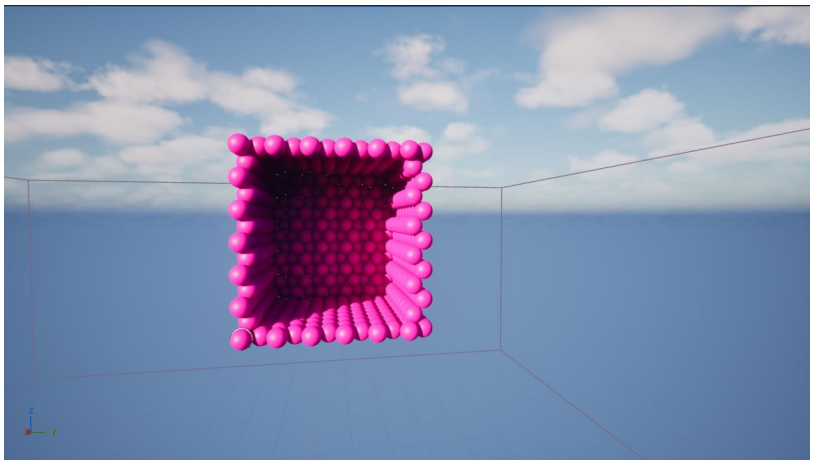

Fig 02.2.1

The HCP Grid Surface Filter performs an operation to an output series of point data that represents the surface of a created HCP voxel points Grid as described in section 1 above. The simplest demonstration of how to implement the HCP Grid Surface filter is illustrated in **Fig 02.2.2** to give the result in the unreal engine editor of **Fig 02.2.1**.

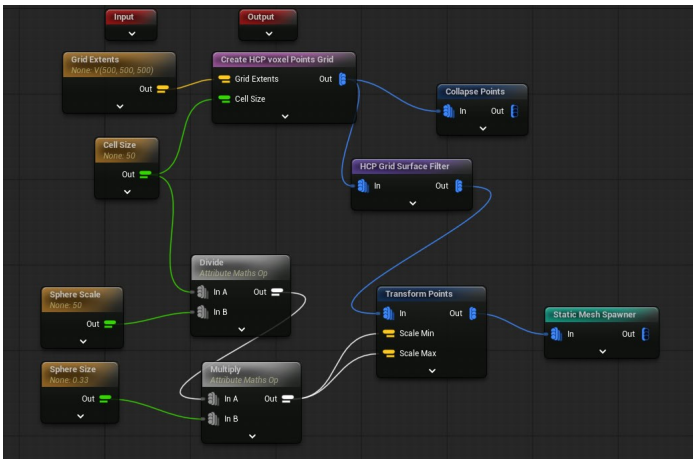

Fig 02.2.2.

A PCG graph file called HCP_Voxel_test should exist within the PCG graphs directory of the unreal engine project Contents directory. Examine this PCG graph in the unreal engine PCG graph editor to make sure it has the same appearance and node links as that given in **Fig 02.2.2**.

It may be that the Create HCP voxel points grid node and HCP Grid Surface Filter node are missing. If this is the case, then press the right button of the mouse and from the node selection widget, search for hcp voxel points grid and HCP Grid Surface Filter, and select them to to add the nodes to the PCG graph editor. Connect all of the node links as given in **Fig 02.1.2**.

Once the user has a PCG graph similar to **Fig 02.2.2**, drag and drop this PCG graph into the unreal engine level displayed in the editor.

The output of the HCP Grid Surface filter does not retain the index number of the input HCP voxel points Grid node for each point that is designated as a surface. Therefore the resultant output cannot be displayed accurately using a Trapezo-rhombic dodecahedral geometric shape for odd and even HCP grid levels. Since the basis of a 3D HCP matrix grid is that of a sphere of the same size as the voxel cell size, the static mesh spawner is set to display a simple spherical mesh scaled to best represent the HCP grid surface.

Using Unreal Engine Custom HLSL Node to create a HCP Grid Shape or surface

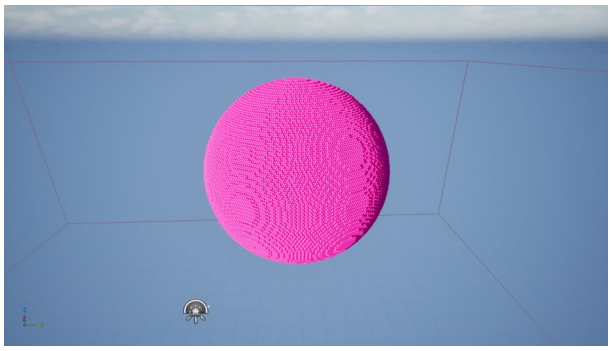

Fig 02.3.1

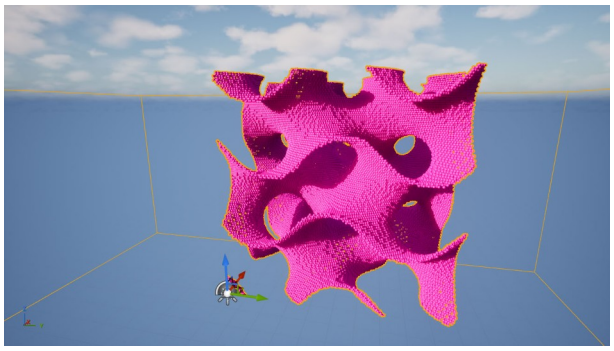

Fig 02.3.2

The most Important element of the HCP application from which the HCP_VPCG was adapted from was to be able to define within the data structure of the HCP voxel grid, a method to define 3D volumes utilising glsl compute shader code. Unreal engine does not use glsl compute shaders, but does have a HLSL compute shader code node that is similar to glsl to perform this task.

A PCG graph file called HCP_HLSL_test should exist within the PCG graphs directory of the unreal engine project Contents directory. Select and examine this PCG graph in the unreal engine PCG graph editor to make sure it has the same appearance and node links as that given in **Fig 02.3.3**.

It may be that the Create HCP voxel points grid node is missing. If this is the case, then press the right button of the mouse and from the node selection widget, search for hcp voxel points grid and select it to to add the node to the PCG graph editor. Connect all of the node links as given in **Fig 02.3.3**.

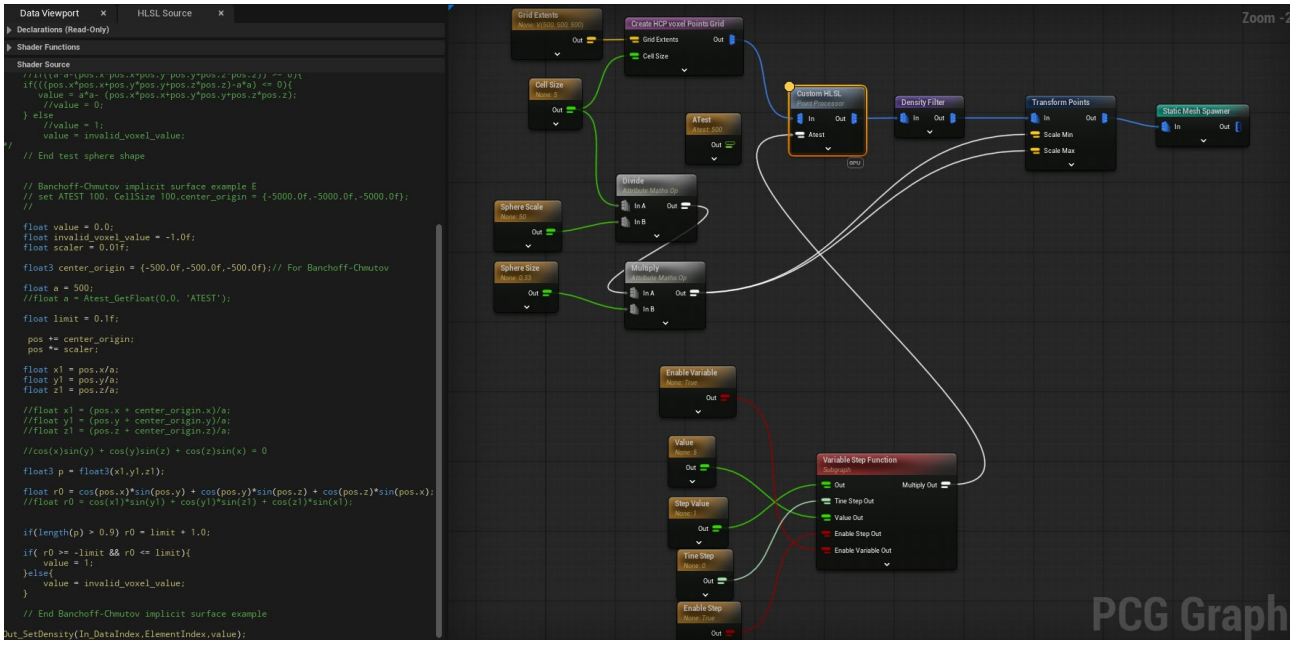

Fig 02.3.3

When it is satisfied that the PCG graph is ready yo be included in an unreal engine level, drag and drop this PCG graph into that unreal engine level in the unreal engine editor.

The critical factor in getting this unreal engine node to function correctly for this example is to make sure that the location of this PCG graph node is set to (0,0,0) for the x,y,z coordinates. To do this, select the HCP_HLSL_test graph node in the outliner of the unreal engine editor, and make sure that within the details panel of the unreal engine editor that the location values within the transform drop down have x,y,z all set to

a value of zero.If this is not done, then the display can look very different to **Fig 02.3.1** or **Fig 02.3.2**, or nothing is displayed at all.This because the grid cell x,y,z coordinates of the hcp voxel points grid depend upon the location of the PCG graph.

This is an emulation of what the original application code from which the HCP_VPCG plugin was adapted from. If one wishes to get the same result, the Voxelise step function sub graph link can be disconnected and the node with the label ATest used in its place.

If one observes, the output of the Custom HLSL is the input into an unreal engine Density Filter. This is because to display the hcp voxel grid 3D volume data as depicted in **Fig 02.3.1** and **Fig 02.3.2**, the value of the point density not equal to the value of zero is considered as a hcp voxel that defines as being part of a 3D volume. Another method can be to create an additional attribute in or from the Create HCP voxel Points Grid data, and filter that. But here, the density attribute is sufficient.

If one looks at the HLSL code editor on the left, the code to generate the data to define the HCP volume shapes is given. A description of how to write HLSL code in for the unreal engine Custom HLSL PCG node is left for the user to pursue if they are unfamiliar with doing so. A good tutorial to start with is given by a youtube video with current URL https://www.youtube.com/watch?v=yAJe0o3BFvM.

However, if one is to look at the HLSL code, there are two sections, with one commented out. The code between Test Sphere Shape and End test sphere shape defines the code to display the hcp spherical shape given in **Fig 02.3.1**. The code between Banchoff-Chmutov implicit surface example and End Banchoff-Chmutov implicit surface example defines the code to display the hcp spherical shape given in **Fig 02.3.2**. By commenting out one section and uncommenting the other, the user can alternate between these two displays.

If the user has the test sphere shape defined to be displayed, if the Atest node is linked to the HLSL ATest Node, changing the value of the Atest node will change the radius of the displayed sphere. Otherwise if the input value of the PCG Subgraph labeled Value is linked to the HLSL ATest Node, changing the value of the Value node will change the radius of the displayed sphere if the boolean value of the Enable Variable node is set to true.

The custom hlsl node opens up enormous potential for being used in the unreal engine. However, as was found when testing the community version of Houdini, there are memory limitations that hinder the size of the defined HCP grid that can be used, and thus limit what this powerful node can be used for. Eg **Fig 02.3.2** is only a small portion or grid size that was able to be defined and displayed in the original application that performed this same task, but not as as polished as in unreal engine.

Create a HCP Surface Points Grid

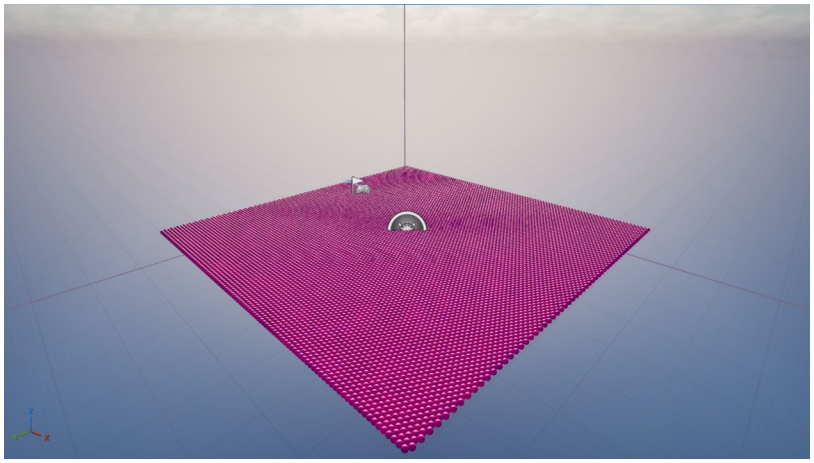

Fig 02.4.1

The second most basic implementation of the HCP_VPCG plugin is to create and display a hcp grid of surface points as in **Fig 02.4.1**.

To create a display of a hcp surface grid similar to **Fig 02.4.1**. a PCG graph file called HCP_Hex_test should exist within the PCG graphs directory of the unreal engine project contents directory. Examine this PCG graph in the unreal engine PCG graph editor to make sure it has the same appearance and node links as that given in **Fig 02.4.2**.

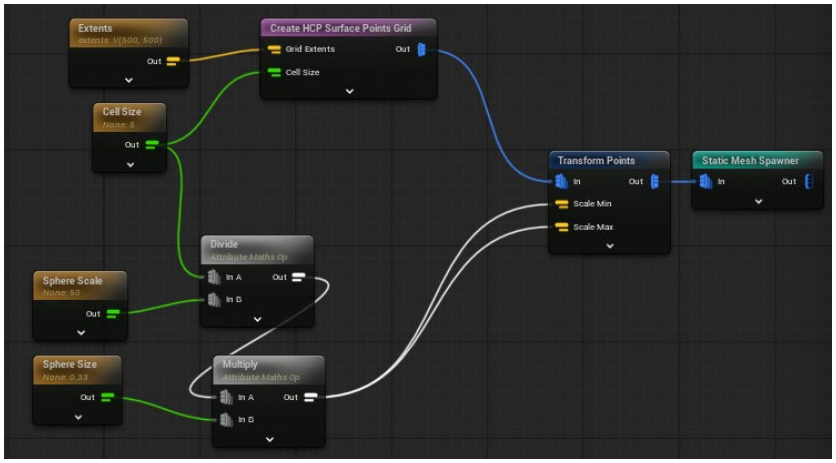

Fig 02.4.2

It may be that the Create HCP surface points grid node is missing. If this is the case, then press the right button of the mouse and from the node selection widget, search for Create HCP surface points grid and select it to add the node to the PCG graph editor. Connect all of the node links as given in **Fig 02.4.2**.

Once it is confirmed that the PCG graph HCP_Hex_test is the same as given in **Fig 02.4.2**. drag and drop the PCG graph HCP_Hex_test into the level, and once done so, a display similar to **Fig 02.4.1** should be present.

If starting from scratch, then the PCG graph of **Fig 02.4.2** needs to be created. The Grid extents node is a float 2 vector node and the CellSize node is a float node. The given grid extents used are 500,500 and cell size 50. By changing these values, the user can see how this generated hcp surface points grid changes, and what limits the unreal engine has on generating such a point grid.

Using Unreal Engine Custom HLSL Node to create a hex grid surface

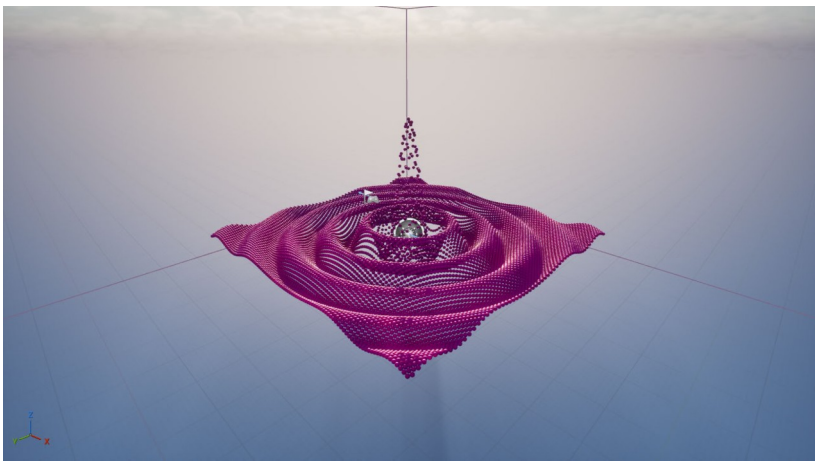

Fig 02.5.1

An Important element of the HCP application from which the HCP_VPCG was adapted from was to be able to define within the data structure of the Hex surface point grid, a method to define 2D surface utilising glsl compute shader code. Unreal engine does not use glsl compute shaders, but does have a HLSL compute shader code node that is similar to glsl to perform this task.

A PCG graph file called HCP_hex_surfce_test should exist within the PCG graphs directory of the unreal engine project contents directory. Select and examine this PCG graph in the unreal engine PCG graph editor to make sure it has the same appearance and node links as that given in **Fig 02.5.2**.

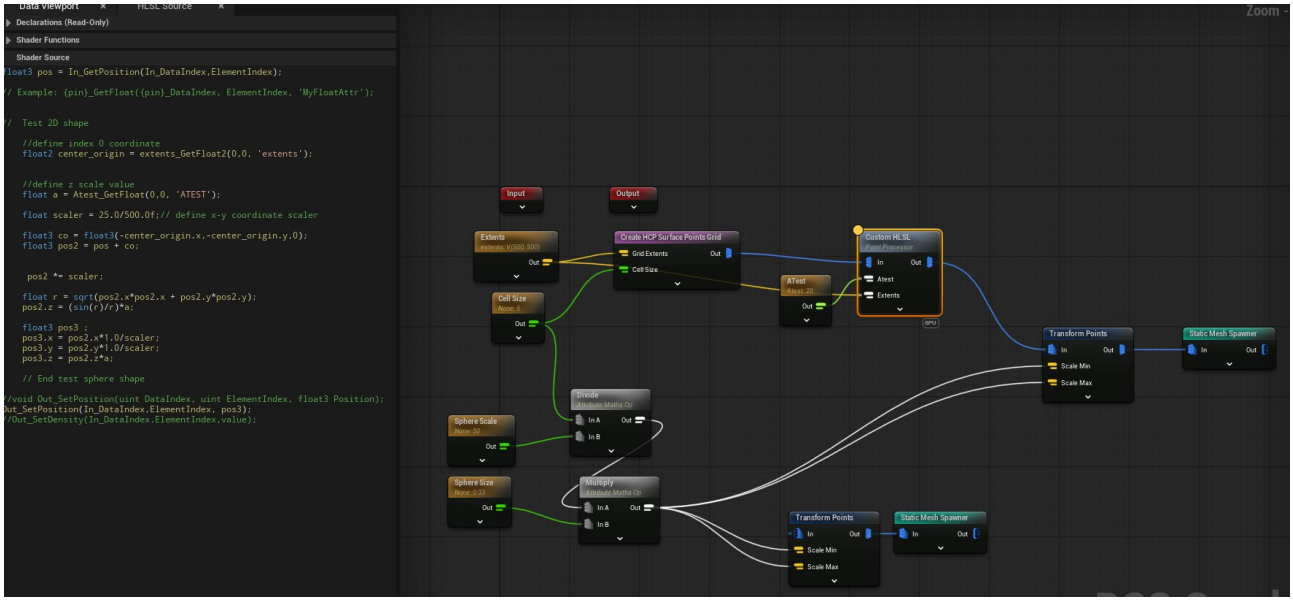

Fig 02.5.2

It may be that the Create HCP surface points grid node is missing. If this is the case, then press the right button of the mouse and from the node selection widget, search for HCP surface points grid and select it to to add the node to the PCG graph editor. Connect all of the node links as given in **Fig 02.5.2**.

When it is satisfied that the PCG graph is ready yo be included in an unreal engine level, drag and drop this PCG graph into that unreal engine level in the unreal engine editor.

The critical factor in getting this unreal engine node to function correctly for this example is to make sure that the location of this PCG graph node is set to (0,0,0) for the x,y,z coordinates. To do this, select the HCP_Hex_surface_test graph node in the outliner of the unreal engine editor, and make sure that within the details panel of the unreal engine editor that the location values within the transform drop down have x,y,z all set to a value of zero. If this is not done, then the display can look very different to **Fig 02.5.1**, or nothing is displayed at all. This because the grid cell x,y,z coordinates of the hcp surface grid depend upon the location of the PCG graph.

If one observes, the output of the Custom HLSL is the input into an unreal engine Transform points node. This is because to display the hcp voxel grid 3D volume data as depicted in **Fig 02.5.1** as a sphere to represent a hex surface data point, the sphere mesh shape used in this example needs to be scaled to match the grid cell size, otherwise the display will have a grid of spheres that are too small be seen, or so large they all overlap.

The input parameters into the Create HCP Surface Points grid are a float2 vector data type to define the grid extents, and a float value to define the resultant hex grid cell size. The output grid is however is a 3D grid of only one z axis level. The z value of this grid is initialised to the value zero that can have this value changed to define a non flat hex grid surface.

If one looks at the HLSL code editor on the left, the code to generate the data to define the HCP surface is given. A description of how to write HLSL code in for the unreal engine Custom HLSL PCG node is left for the user to pursue if they are unfamiliar with doing so. A good tutorial to start with is given by a youtube video with current URL https://www.youtube.com/watch?v=yAJe0o3BFvM.

Within this code, the z value is designated a value that defines the surface of the hex grid, and it is a float3 vector of that defines the output surface of this node. Of course the user can create another attribute to take the place of the z coordinate of a 2D surface, but this is the most logical and consistent method.

The unreal engine PCG graph node ATest is a scale factor that determines the value or height of the z coordinate. Changing this value will scale the surface Z value by the value entered.

As with the HCP voxel points grid, the custom hlsl node opens up enormous potential for being used in the unreal engine.